coreclaw-nonfunctional-requirements

CoreClaw 非機能要件定義書

本書は、CoreClaw の現行コードを根拠として、運用品質、保守性、安全性、可用性などの非機能要件を整理したものである。対象機能の詳細は qiita/coreclaw-functional-requirements.md を前提とする。

1. 目的

- CoreClaw を継続運用するうえで必要な品質条件を定義する。
- 実装済みの仕組みを踏まえ、今後の改善で守るべき設計原則を明文化する。
- テスト、運用、リリース、障害対応の基準を揃える。

2. 適用範囲

- Web UI
- Node.js サーバー API / WebSocket
- Docker コンテナ実行基盤
- SQLite 永続化
- 設定、スキル、Marketplace、MCP、GitHub 認証と同期

3. 非機能要件

3.1 可用性

ID	要件	内容
NFR-01	継続稼働	システムは、Web UI、API、WebSocket を単一プロセスで継続提供できること。
NFR-02	再起動耐性	システムは、再起動後に設定、実験、会話、履歴、登録グルー

ID	要件	内容
		プ、セッションを復元できること。
NFR-03	切断復元	システムは、WebSocket 切断後に再接続したクライアントへ進行中タスク状態を再配信できること。
NFR-04	Graceful 停止	システムは、API 経由の停止時に異常終了ではなく制御された終了シーケンスを提供すること。

3.2 性能

ID	要件	内容
NFR-05	応答性	システムは、通常操作に対して UI が即時に反応し、長時間処理は非同期で扱うこと。
NFR-06	ストリーミング性能	システムは、長文応答や実行ログを全文完了まで待たず逐次表示できること。
NFR-07	履歴取得制御	システムは、メッセージや活動ログ取得時に件数制御を行い、過大レスポンスを避けること。
NFR-08	長時間実行制御	システムは、コンテナ実行に対してタイムアウトを設定し、無制限実行を避けること。

3.3 拡張性

ID	要件	内容
NFR-09	Skill 拡張性	システムは、skills ディレクトリへ追加されたスキルを取り込める構造であること。
NFR-10	MCP 拡張性	システムは、任意の stdio/SSE ベース MCP を追加定義できること。
NFR-11	Marketplace 拡張性	システムは、公式ソースに加えてカスタム GitHub リポジトリを Marketplace ソースとして利用できること。
NFR-12	チャネル拡張性	システムは、チャネルをインターフェースで抽象化し、新規チャネル追加を可能にすること。

3.4 保守性

ID	要件	内容
NFR-13	責務分離	システムは、Web サーバー、DB、実験管理、コンテナ実行、認証、更新処理を分離した構成を維持すること。
NFR-14	設定集約	システムは、主要設定を data/settings.json および環境変数から一元管理できること。
NFR-15	スキーマ移行	システムは、DB マイグレーションをコードで実行し、既存データを保持したまま更新できること。
NFR-16	追跡可能性	システムは、タスク履歴、活動ログ、エラーを通じて処理経路を追跡できること。

3.5 セキュリティ

ID	要件	内容
NFR-17	コンテナ分離	エージェント実行はホスト直実行ではなく Docker コンテナで隔離すること。
NFR-18	認証情報保護	GitHub Token などの秘密情報は、設定取得時にマスクして表示すること。
NFR-19	認証情報非直渡し	コンテナへ GitHub Token を直接渡さず、認証プロキシ経由で GitHub API へアクセスさせること。
NFR-20	入力値サニタイズ	実験 ID、スキル名、アップロードファイル名、パス入力などは不正文字や危険なパスを除去すること。
NFR-21	マウント制御	コンテナへの追加マウントは許可リストで制御されること。
NFR-22	スキル検査	スキル導入時または管理時に危険なコードやコマンドを検出するスキャン手段を提供すること。
NFR-23	重複除去	MCP 設定は同名重複を排除し、競合した定義を保持しないこと。

3.6 データ保全性

ID	要件	内容
NFR-24	永続化	実験、会話、スケジュール、履歴は再起動で消失しないこと。
NFR-25	ファイル完全性	実験成果物は実験専用ディレクトリに保持されること。
NFR-26	出力保護	ZIP 取込時に危険パスや不正展開を避けること。
NFR-27	変更反映整合性	設定変更後は以降の処理に反映されること。

3.7 観測性

ID	要件	内容
NFR-28	ログ出力	システムは、主要操作や障害をログ出力できること。
NFR-29	活動イベント可視化	ユーザーは、エージェントの処理途中経過を UI から確認できること。
NFR-30	状態配信	サーバーは、タスク一覧や状態変化を WebSocket で配信できること。
NFR-31	認証状態可視化	Copilot 認証状態を UI で把握できること。

3.8 ユーザビリティ

ID	要件	内容
NFR-32	一貫 UI	実験管理、設定、成果物閲覧、検索を一つの SPA で完結できること。
NFR-33	低学習コスト	主要操作はボタンまたはモーダル中心で提供し、CLI を必須にしないこと。
NFR-34	日本語対応	日本語 UI と日本語出力を扱えること。
NFR-35	多言語出力	最終応答の出力言語を設定可能であること。
NFR-36	補助入力	音声入力を補助的に利用できること。

3.9 運用性

ID	要件	内容
NFR-37	環境構築容易性	

ID	要件	内容
		Node.js、Docker、GitHub Token を前提に再現可能なセットアップ手順を提供すること。
NFR-38	更新容易性	バージョン確認と更新実行をアプリ内から行えること。
NFR-39	Dirty 状態考慮	更新時にローカル変更の有無を判定し、危険な自動更新を抑制できること。
NFR-40	ポート可変性	Web サーバーと認証プロキシのポートは環境変数で変更可能であること。

3.10 互換性

ID	要件	内容
NFR-41	ブラウザ利用	一般的なモダンブラウザで UI が利用できること。
NFR-42	Linux/WSL/macOS 考慮	コンテナ接続やホストゲートウェイの扱いは主要開発環境差異を考慮すること。
NFR-43	Unicode 対応	日本語を含む Unicode ファイル名およびテキストを扱えること。

4. 品質目標

- 長時間タスクでも UI が固まらないこと。
- タスク失敗時に利用者が原因を追いやすいこと。
- 追加スキルや MCP を導入しても既存実験の安全性を著しく損なわないこと。
- 設定変更や再起動を伴う運用を利用者が UI 主体で実施できること。

5. 運用上の前提

- GitHub Copilot 利用権と GitHub Token が利用可能であること。
- Docker 実行環境が起動していること。
- data、groups、skills 配下へ読み書き可能であること。
- 外部 STT や GitHub Marketplace 利用時はネットワーク疎通があること。

6. 今後の補強候補

- 定量的な性能目標値の設定
- バックアップ/リストア運用要件の明確化
- 権限制御、監査ログ、ユーザー分離の強化
- 障害復旧手順、監視設計、SLA/SLO の明文化